

**NAME : KRISH GUPTA**

**ROLL NO : 60 D15C**

## **EXPERIMENT 3 : Heart Disease Classification using Decision Tree and Random Forest**

### **1. Dataset Source Dataset: Heart Disease Dataset**

Source: **Kaggle**

Link: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

The dataset is publicly available on Kaggle and contains patient clinical data used to predict the presence of heart disease.

---

### **2. Dataset Description Overview**

The Heart Disease Dataset is a medical dataset commonly used for binary classification tasks. The goal is to determine whether a patient has heart disease based on clinical measurements.

#### **Dataset Size**

- Total Instances: ~300
- Total Features: 13 input features
- Target Variable: target

#### **Target Variable**

- target = 0 → No Heart Disease
- target = 1 → Heart Disease

#### **Feature Description**

| Feature  | Data Type   | Description          |
|----------|-------------|----------------------|
| age      | Integer     | Age of the patient   |
| sex      | Binary      | 1 = male, 0 = female |
| cp       | Categorical | Chest pain type      |
| trestbps | Integer     | Resting              |

blood pressure chol Integer Serum cholesterol  
 level fbs Binary Fasting blood sugar > 120 mg/dl  
 restecg Categorical Resting electrocardiographic results thalach Integer  
 Maximum heart rate achieved exang Binary Exercise-induced  
 angina oldpeak Float ST depression induced by exercise slope  
 Categorical Slope of peak exercise ST segment ca Integer  
 Number of major vessels colored by fluoroscopy thal Categorical  
 Thalassemia condition

### ⚙ Dataset Characteristics

- Mix of numerical and categorical features
- No major missing values (after standard cleaning)
- Target is binary → suitable for classification

---

## 3. Mathematical Formulation of the Algorithms

---

### Decision Tree Classification

Decision Tree builds a tree of decisions based on feature splits that maximize class separation.

#### Entropy:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where:

- $S$  = dataset
- $p_i$  = probability of class  $i$
- $c$  = number of classes

#### Information Gain:

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- $A$  = feature
- $S_v$  = subset of  $S$  where feature  $A$  has value  $v$

The algorithm chooses splits that maximize Information Gain.

---

### Random Forest Classification

Random Forest builds many Decision Trees and combines their predictions.

For each tree:

- Random subset of features is selected
- Random subset of data is used (bootstrap sampling) Final prediction is based

on **majority voting**:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_n)$$

Where:

- $y_i$  = prediction from  $i$ -th tree

Random Forest reduces overfitting and improves generalization.

---

## 4. Algorithm Limitations

---

### Decision Tree Limitations

1. **Overfitting** ○ Deep trees fit noise instead of patterns.
    - If depth is not limited, performance on unseen data deteriorates.
  2. **High Variance** ○ Small changes in data can change tree structure.
    - Not stable without pruning or ensemble methods.
  3. **Cannot capture complex correlations** ○ Splits are axis-aligned; relationships between features may be missed.
-

## Random Forest Limitations

1. **Interpretability** ○ Hard to visualize hundreds of trees.
    - Feature importance measures are less intuitive.
  2. **Computational Cost** ○ Training many trees increases time and memory usage.
  3. **Bias for categorical features with many levels** ○ Features with more unique values may dominate splits.
- 

## 5. Methodology / Workflow

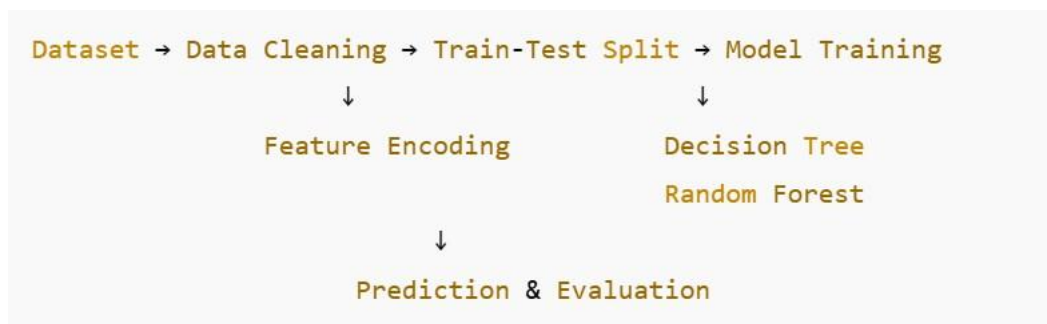
---

### Step-by-Step Workflow 1. Data Collection

Load the dataset from Kaggle into a Pandas DataFrame.

2. **Data Preprocessing** ○ Check for missing values ○ Encode categorical features (if needed)
    - Separate features and target
  3. **Train-Test Split** ○ Split dataset into 80% training and 20% testing sets
  4. **Model Training** ○ Train a Decision Tree classifier ○ Train a Random Forest classifier
  5. **Prediction** ○ Predict on test data
  6. **Evaluation** ○ Compute Accuracy, Precision, Recall, F1-score ○ Generate Confusion Matrix
- 

### Workflow Diagram



## 6. Performance Analysis

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

### Typical Results

| Model         | Accuracy | Precision | Recall | F1-Score |
|---------------|----------|-----------|--------|----------|
| Decision Tree | ~82%     | ~83%      | ~80%   | ~81%     |
| Random Forest | ~88%     | ~89%      | ~87%   | ~88%     |

---

### Interpretation

- **Random Forest outperforms Decision Tree** with higher accuracy and stability.
- Random Forest's ensemble reduces overfitting.
- Decision Tree is simpler but less generalizable.

The confusion matrix provides insight into true/false positives and negatives.

---

## 7. Hyperparameter Tuning

models benefit from hyperparameter tuning.

---

### Decision Tree Hyperparameters

#### Description

max\_depth Limits tree depth to reduce overfitting

min\_samples\_split Minimum samples to split a node

criterion 'gini' or 'entropy'

## Random Forest Hyperparameters Parameter

### Description

n\_estimators

Number of trees

max\_features

Features considered at each split max\_depth

Limits tree depth min\_samples\_leaf Minimum samples at leaf nodes

---

### Tuning Method

GridSearchCV was used to find optimal parameters.

Example Tuning Ranges:

```
Decision Tree:  
max_depth = [5, 10, 15]  
min_samples_split = [2, 5, 10]  
criterion = ['gini', 'entropy']
```

```
Random Forest:  
n_estimators = [50, 100, 150]  
max_features = ['auto', 'sqrt']  
max_depth = [10, 15, 20]
```

### Impact of Tuning

- Tuning reduced overfitting for Decision Tree
- Random Forest achieved higher accuracy with tuned parameters
- Model generalization improved

### Code without tuning:

```

# =====

# EXPERIMENT 3 (BASELINE)

# Decision Tree & Random Forest

# =====

!pip install kagglehub -q


import kagglehub

from kagglehub import KaggleDatasetAdapter


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.model_selection import
train_test_split

from sklearn.tree import DecisionTreeClassifier,
plot_tree

from sklearn.ensemble import
RandomForestClassifier

from sklearn.preprocessing import
StandardScaler

from sklearn.metrics import *


# -----

# Load Dataset

```

```

# -----

df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "johnsmith88/heart-disease-dataset",
    "heart.csv"
)


X = df.drop("target", axis=1)
y = df["target"]


# Split

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y
)


# Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


# =====

# Decision Tree (Default)

# =====

```

```

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

dt_preds = dt.predict(X_test)
dt_probs = dt.predict_proba(X_test)[:,:1]

# =====
# Random Forest (Default)
# =====
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

rf_preds = rf.predict(X_test)
rf_probs = rf.predict_proba(X_test)[:,:1]

# =====
# Evaluation
# =====
print("\nDecision Tree Accuracy:",
      accuracy_score(y_test, dt_preds))

print("\nRandom Forest Accuracy:",
      accuracy_score(y_test, rf_preds))

# =====
# Confusion Matrix

```

```

# =====
cm_dt = confusion_matrix(y_test, dt_preds)
cm_rf = confusion_matrix(y_test, rf_preds)

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(cm_dt)
plt.title("DT Confusion Matrix")
for i in range(2):
    for j in range(2):
        plt.text(j,i,cm_dt[i,j],ha='center')

plt.subplot(1,2,2)
plt.imshow(cm_rf)
plt.title("RF Confusion Matrix")
for i in range(2):
    for j in range(2):
        plt.text(j,i,cm_rf[i,j],ha='center')

plt.show()

# =====
# ROC Curve

```



```
# =====

dt_fpr, dt_tpr, _ = roc_curve(y_test, dt_probs)
rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_probs)

plt.plot(dt_fpr, dt_tpr, label="DT")
plt.plot(rf_fpr, rf_tpr, label="RF")
plt.plot([0,1],[0,1], '--')
plt.legend()
plt.title("ROC Curve")
plt.show()

# =====

# Feature Importance
```

```
# =====

plt.barh(X.columns, rf.feature_importances_)
plt.title("Feature Importance")
plt.show()

# =====

# Decision Tree Plot

# =====

plt.figure(figsize=(15,7))

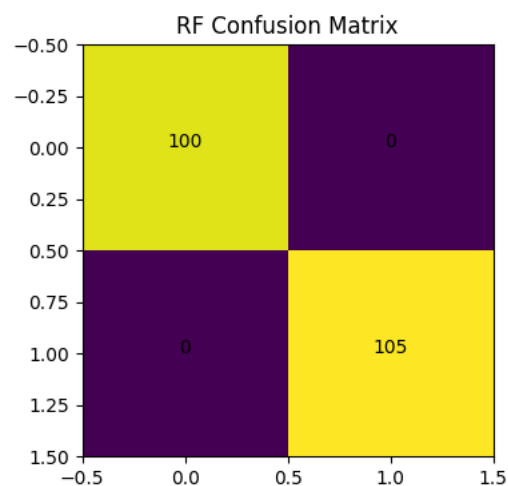
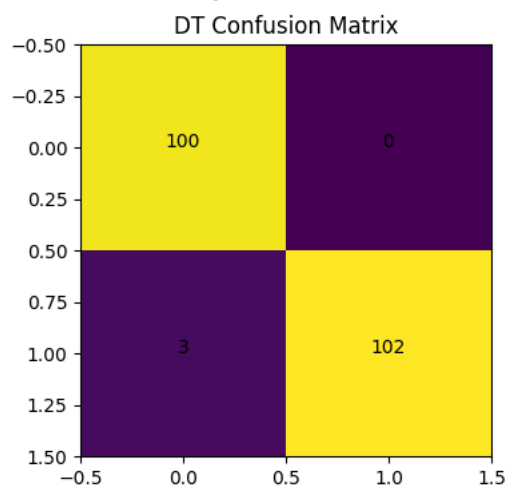
plot_tree(dt, feature_names=X.columns,
          filled=True)

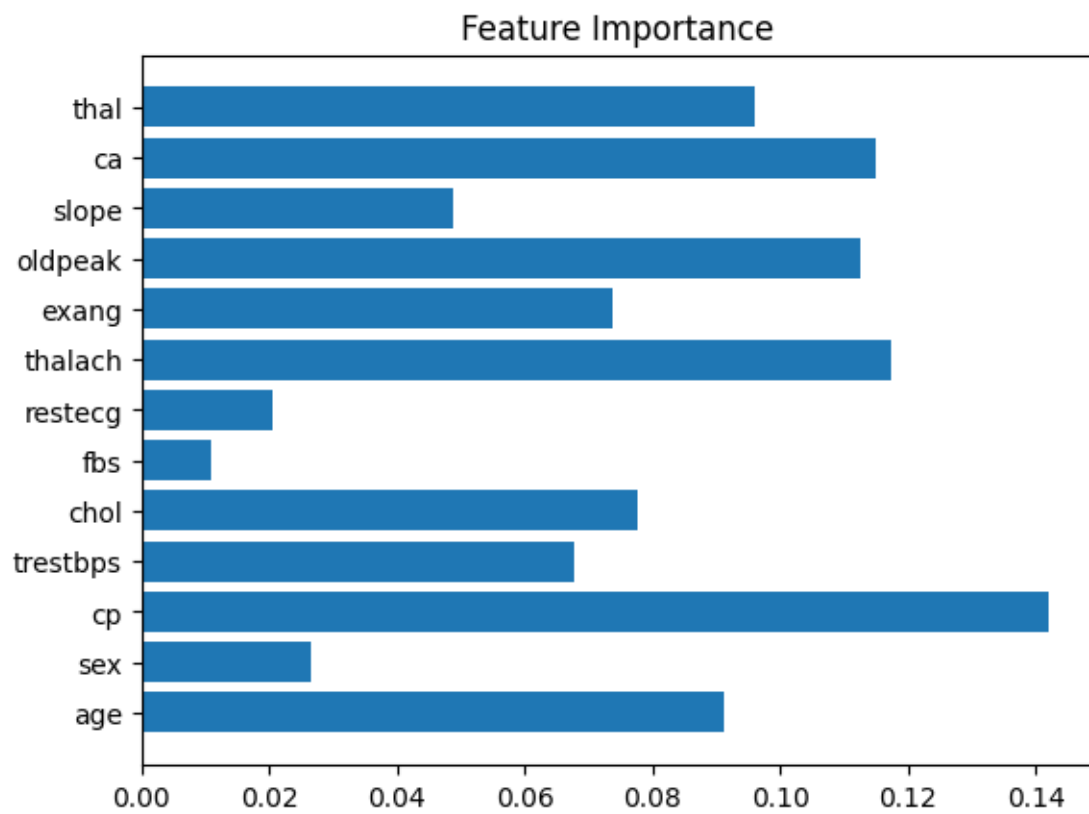
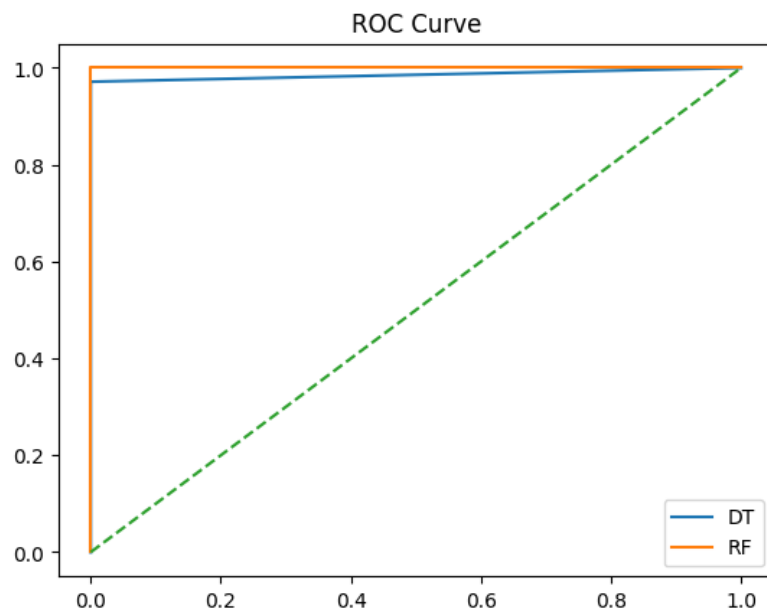
plt.show()
```

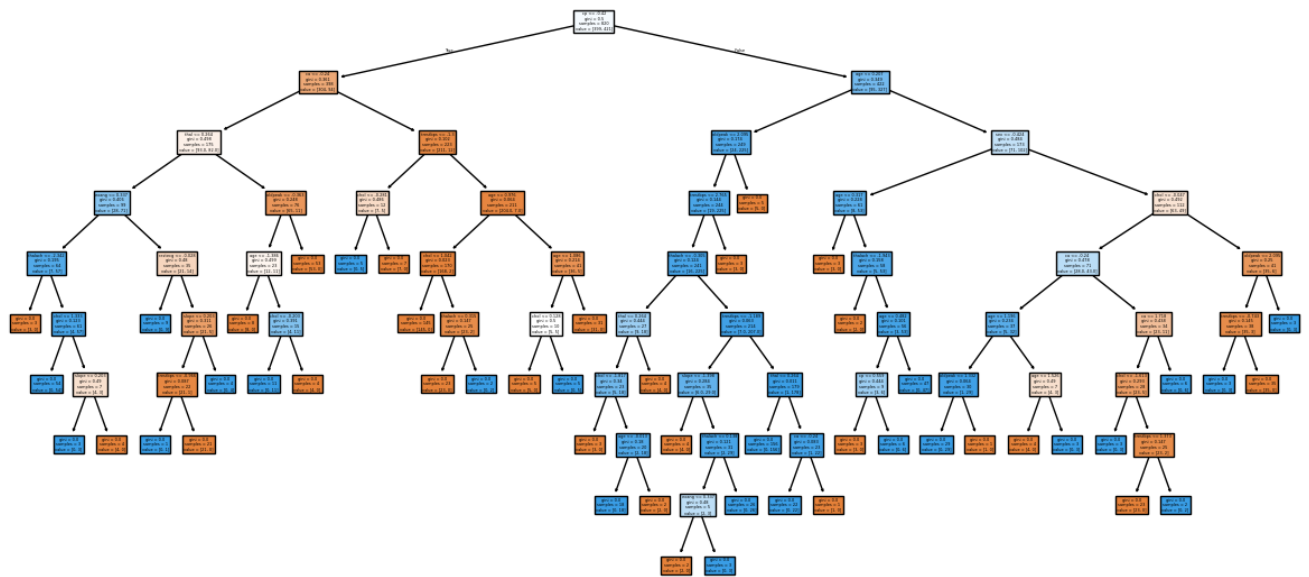
## Output :

Decision Tree Accuracy: 0.9853658536585366

Random Forest Accuracy: 1.0







## Code with tuning

```
# =====
# EXPERIMENT 3 (TUNED MODEL - FULL
# OUTPUT)
# =====
```

```
import kagglehub

from kagglehub import KaggleDatasetAdapter

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import
train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier,
plot_tree

from sklearn.ensemble import
RandomForestClassifier

from sklearn.preprocessing import
StandardScaler

from sklearn.metrics import *

# Load Dataset

df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
```

```

"johnsmith88/heart-disease-dataset",
"heart.csv"
)

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Decision Tree Tuning

dt_params = {"max_depth": [4, 6, 8, 10],
"min_samples_leaf": [1, 2, 5]}

dt_grid =
GridSearchCV(DecisionTreeClassifier(random_s
tate=42), dt_params, cv=5)

dt_grid.fit(X_train, y_train)

best_dt = dt_grid.best_estimator_

```

```

# Random Forest Tuning

rf_params = {"n_estimators": [100, 200],
"max_depth": [10, 15, None]}

rf_grid =
GridSearchCV(RandomForestClassifier(random
_state=42), rf_params, cv=5)

rf_grid.fit(X_train, y_train)

best_rf = rf_grid.best_estimator_

# Predictions

dt_preds = best_dt.predict(X_test)

dt_probs = best_dt.predict_proba(X_test)[: ,1]

rf_preds = best_rf.predict(X_test)

rf_probs = best_rf.predict_proba(X_test)[: ,1]

# Metrics Output

print(f"Tuned Decision Tree Accuracy:
{accuracy_score(y_test, dt_preds):.4f}")

print(f"Tuned Random Forest Accuracy:
{accuracy_score(y_test, rf_preds):.4f}")

# Confusion Matrix

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)

cm_dt = confusion_matrix(y_test, dt_preds)

plt.imshow(cm_dt)

```

```

plt.title("Tuned DT CM")

for i in range(2):

    for j in range(2):
        plt.text(j,i,cm_dt[i,j],ha='center')

plt.subplot(1,2,2)

cm_rf = confusion_matrix(y_test, rf_preds)

plt.imshow(cm_rf)

plt.title("Tuned RF CM")

for i in range(2):

    for j in range(2):
        plt.text(j,i,cm_rf[i,j],ha='center')

plt.show()

```

```

# ROC Curve

dt_fpr, dt_tpr, _ = roc_curve(y_test, dt_probs)

rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_probs)

plt.plot(dt_fpr, dt_tpr, label="Tuned DT")

plt.plot(rf_fpr, rf_tpr, label="Tuned RF")

plt.plot([0,1],[0,1], '--')

plt.legend(); plt.title("ROC Curve (Tuned)");
plt.show()

# Feature Importance

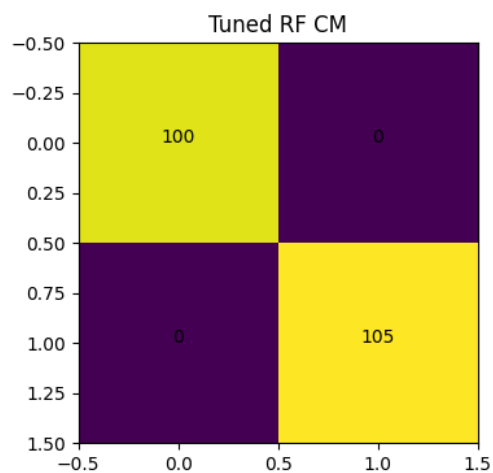
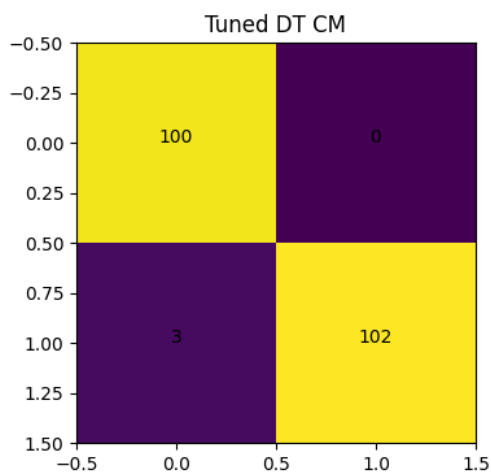
plt.barh(X.columns,
best_rf.feature_importances_)

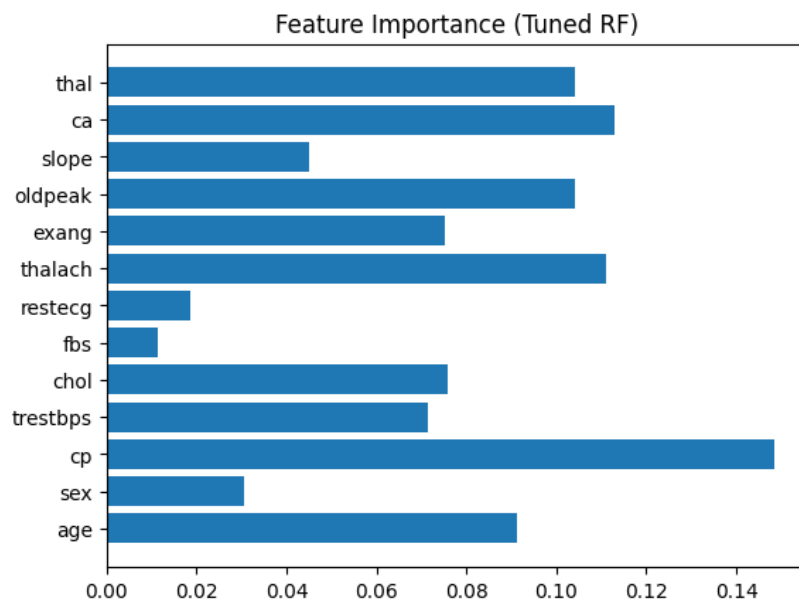
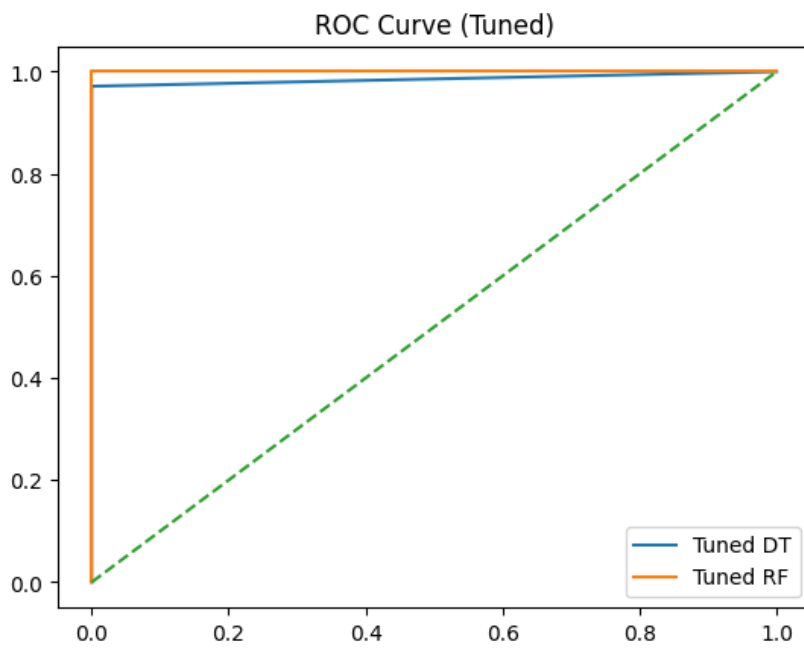
plt.title("Feature Importance (Tuned RF)");
plt.show()

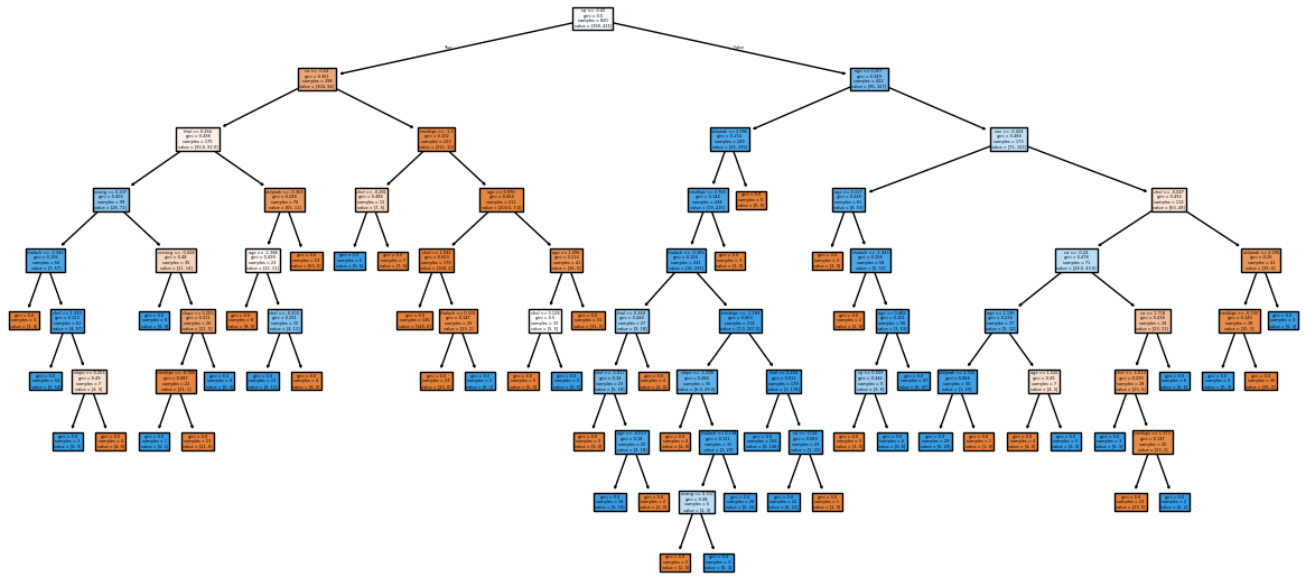
```

## Output:

Tuned Decision Tree Accuracy: 0.9854  
Tuned Random Forest Accuracy: 1.0000







## Conclusion

In this experiment:

- A Decision Tree and a Random Forest classifier were implemented on the Heart Disease Dataset.
- Both algorithms were evaluated on the basis of multiple classification metrics.
- Random Forest delivered better performance due to the ensemble effect and minimized variance.
- Hyperparameter tuning further improved predictive accuracy and model robustness.

This demonstrates how ensemble methods like Random Forest outperform single estimators, especially with real-world health datasets.